

CpSc 418

Homework 6

Due: Dec. 8, 2025, 11:59pm

Early Bird: Dec. 6, 2025, 11:59pm

Please submit your solution using handin:

```
handin cs-418 hw6
```

Your solution should have these files:

```
any(hw6.pdf, hw6.txt)
```

```
hw6_lib.h
```

```
hw6.cu
```

```
Makefile
```

```
hw6_mem.cu
```

```
hw6_lib.cu
```

```
hw6_mem.h.
```

The Questions

0. Collaborators:

Extra Credit:

- (a) Include your prompts to AI assistants as described above.
- (b) Tell us why you didn't use an AI assistant or didn't find it helpful.
- (c) Write a few sentences that you think would be the best advice you could give to a student starting this assignment based on your experience of completing it.
- (d) Identify a question that you thought was particularly helpful or pointlessly tedious, and give us a few sentences saying why.
- (e) Write something else that you think is worthy of extra-credit. Mark it clearly as your response to the **Extra Credit** part of this question.

1. Coalesced memory references:

This is a read and run-code problem. You'll get to write code in the subsequent problems.

Download the source files {hw6,hw6_lib, hw6_mem}.{cu,h} and Makefile. Build the mem program:

```
make mem
```

Please do this on one of the linXX machines (i.e. lin01, lin02, ..., or lin25).

If you run

```
mem
```

it will run a kernel with parameters where all memory references are coalesced. OTOH,

```
./mem - - - false
```

it will the kernel where all memory references are non-coalesced. See the comments at the beginning of hw6.cu for the many options you can use.

- Explain why the kernel with 'coalesced' set to 'true' has all of its memory references coalesced. This requires reading the code. I'll explain a few key points below.
- Explain why the kernel with 'coalesced' set to 'false' has nearly all of its memory references not coalesced. Again, read the code and my notes below.
- Why do we get more accurate estimates when n (the number of int's in the main data array) is large? Feel free to experiment with different values for n . I recommend using power-of-two values. It is sufficient to make the observation and explain the trends in terms of the architecture features.
Hint: if you want to dive deeper into this problem, you can use these measurements to estimate the L2 cache size and miss penalty.
- What is the memory bandwidth achieved when the memory references are coalesced?
- What is the memory bandwidth achieved when the memory references are not coalesced?
- How much longer does it take a warp to do WarpSize global memory reads when references are not coalesced than when they are coalesced.

Notes about collisions:

The `gmem_fetch` kernel has each thread perform `n_read` reads from the global memory. Let

```
i = blockDim.x * blockIdx.x + threadIdx.x
```

be the global index of the thread. Thread i first reads from array index i , then $(i+stride[i]) \% n$, then $(i+2*stride[i]) \% n$, and so on.

The k^{th} read (with k starting at 0) is from

```
ar[i + k*stride[i]].
```

To get all of the accesses coalesced, the stride array is initialized so that `stride[i]=gridDim.x*blockDim.x` (i.e. the total number of threads).

Why does this produce coalesced memory accesses? (i.e. Q1.a).

To get the accesses non-coalesced (with high probability), `stride[i]` is initialized to a random integer with $0 \leq stride[i] < n$. Why does this produce non-coalesced reads (with high probability)?

We would get even better randomization if for each read, each thread independently chose a new random address in $0..n-1$. But, pseudo random number generation is much slower than a single global memory access; so this would make measurements difficult. The method I used is, empirically, just as good and keeps the kernel memory bound.

2. Shared memory collisions:

In this problem, you will write a new kernel, similar to the `gmem_fetch` kernel in `hw6_lib.cu`. Call this kernel `smem_collide`. With this kernel each block should perform a collaborative read to load data from global memory into shared memory. Each thread should then perform a large number of reads from the shared memory. These reads should use indexing methods similar to those in `gmem_fetch` so the compiler can't figure out the access pattern at compile time (it should depend on the parameters used when running the program).

- (a) Find an access pattern so that if `n_read` is large enough, every thread in the block will eventually read from every location loaded into the shared memory and these reads incur no shared memory collisions. Describe what property an access pattern needs to have to ensure that there are no shared-memory collision. Explain why your access pattern satisfies this property. Your description and explanation should be similar in level of detail to the explanatory note for Q1.
- (b) Find an access pattern so that if `n_read` is large enough, every thread in the block will eventually read from every location loaded into the shared memory and these reads incur the maximum shared-memory collision penalty. I.e. you are trying to make the code as **slow** as possible. Describe what property an access pattern needs to have to maximize the shared-memory collision penalty. Explain why your access pattern satisfies this property. Your description and explanation should be similar in level of detail to the explanatory note for Q1.
- (c) Implement your kernel to perform your access patterns from Q2.a and Q2.b. Report the execution times you measure. From this data, what is the execution time penalty for a warp to perform a shared memory read with maximum collisions.

3. Random accesses of the shared memory:

Consider a kernel that makes random accesses to the shared memory. What is the expected collision penalty for each read performed by a warp?

This amounts to picking `WarpSize` integers uniformly and independently at random from `{0, 1, ..., WarpSize-1}`, where `WarpSize=32`. Find the value that occurs the most times. For example for one trial, you might find that the value 19 occurs 7 times, and all other values occur at most 7 times. What is the expected number of occurrences for the value that occurs the most times? For the one sample described above, this number is 7.

There are (at least) three ways you can solve this:

- (a) Write a simulation in any "reasonable" programming language: C, C++, CUDA, Erlang, Java, Javascript (but why?), Julia, Matlab, Python (I won't tell), R, etc. Your code should be clear with comments that explain the key ideas. Please don't write a comment for each line that just says in English what is already said more clearly in the code. What estimate do you get?
- (b) Set-up a recurrence to compute the answer, and write code to compute the value for the recurrence (same guidelines as for approach 'a').
- (c) Set-up the recurrence and find a nice simple formula to give it a closed form. I haven't found one, but I haven't given it lots of time. We'll give some extra credit for a clear and correct derivation.

4. Debug the textbook:

I'll post my implementation of the convolution kernel described in the book. Compare my version to the one in the book. Spot the differences. Explain the impact of each change. This problem isn't finalized yet.



Unless otherwise noted or cited, the questions and other material in this homework problem set is copyright 2025 by Mark Greenstreet and are made available under the terms of the Creative Commons Attribution 4.0 International license <http://creativecommons.org/licenses/by/4.0/>